Повторення прошивки перед контрольною

Урок 4 — 2МАК, SST-INF-01c — Входи і виходи, оператори, конструкції, функції

12/11/2026

Примітка

Цей конспект збирає **повторення** прошивки (*firmware*), зроблене в класі перед контрольною (*verifica*). Це **повторення**, а не нове пояснення: подробиці — у конспектах **A01**, **A01bis**, **A02** і **A03**. До контрольної вчимо **ці чотири і цей**. Блокнот NotebookLM класу тепер містить і A03; кожен може створити й **власний блокнот** з конспектами всіх предметів.

1 Входи і виходи: `pinMode`, `digitalWrite`, `digitalRead`

1.1 Вибрати напрямок: `pinMode`

Визначення

Щоб сказати Arduino, чи є вивід (*piedino*) **входом** чи **виходом**, використовують **`pinMode`**: у дужках **номер виводу**, кома і **напрямок**, `INPUT` або `OUTPUT`, написаний **великими літерами**. Закриваємо дужку і ставимо **крапку з комою**.

```
pinMode(6, OUTPUT);
```

На **Arduino Uno** цифрові виводи, якими ми користуємося, — від **2 до 13** (0 і 1 потрібні для послідовного зв'язку). **Arduino Mega** має набагато більше цифрових виводів (54, з номерами від 0 до 53): команда та сама, змінюється лише номер, який можна написати.

1.2 Записати вихід: `digitalWrite`

Визначення

Якщо вивід є виходом, **`digitalWrite`** вмикає або вимикає його: у дужках **номер виводу**, кома і **`HIGH`** (увімкнено) або **`LOW`** (вимкнено), завжди великими літерами.

```
digitalWrite(6, HIGH);
```

1.3 Прочитати вхід: `digitalRead`

Визначення

Якщо вивід є входом, **`digitalRead`** його **читає**: у дужках **лише номер виводу**. Зазвичай прочитане значення **зберігають у змінній**.

```
pinMode(7, INPUT);
int x = digitalRead(7);
```

Увага

Усередині `digitalRead` **не пишуть** `INPUT`: напрямок ми вже вибрали через `pinMode`. Тут **один параметр** — номер виводу; чи вивід високий, чи низький, скаже сам `digitalRead`. З цими трьома командами Arduino може **«говорити»** (виходи) і **«чути»** (входи).

2 Знак рівності: присвоєння і порівняння

Визначення

У C знак `=` — це **присвоєння** (*assegnazione*): він **копіює значення справа** у **змінну зліва**. У `x = digitalRead(7);` прочитане значення потрапляє в `x`; у `a = 5 + 3;` результат 8 потрапляє в `a`.

Увага

У C знак рівності не можна перевертати. У математиці $5 + 3 = 8$ і $8 = 5 + 3$ — те саме. У C `5 + 3 = a;` — це **помилка**: зліва від знака рівності має бути **змінна**, а ім'я змінної **не може містити символів** на кшталт + (дозволене лише підкреслення _) і **не може починатися** з **цифри**. «Скопіювати 8 у 5+3» не має сенсу.

Визначення

Подвійний знак рівності `==` — це інше: це **порівняння**, **питання** («чи вони рівні?»). Він відповідає «істина» або «хиба» і **не змінює** жодної змінної. Тому `a == b` і `b == a` — це **одне й те саме питання**.

3 Оператори

Оператори — це символи, які виконують певну обробку даних.

3.1 Арифметичні оператори і остача від ділення

Є базові дії: +, -, * (множення), / (ділення). Крім того, є символ **%**, який у нашому світі називається не «відсоток», а **модуль** (*modulo*).

Визначення

Модуль % дає **остачу ділення цілих чисел**.

Приклад

- `5 / 2` **для цілих дорівнює 2**, а не 2,5: ділення двох цілих не має коми. Справді, $2 \cdot 2 = 4$, і до 5 **не вистачає 1**.
- `5 % 2` **дорівнює 1**: це якраз та остача.
- `100 % 49` **дорівнює 2**: 49 вміщається в 100 **двічі** ($2 \cdot 49 = 98$), і **залишається** 2.

Увага

Типова помилка. На питання «скільки буде 5 поділити на 2?» багато хто відповідає 2,5 (ділення з комою) або 1 (це **модуль**, тобто відповідь на інше питання). Для **цілих** `5 / 2` дорівнює **2**, а `5 % 2` дорівнює **1**.

Примітка

Парне чи непарне. Модуль 2 одразу каже, чи число **парне**, чи **непарне**: якщо `n % 2` дорівнює **0**, число парне, якщо **1** — непарне.

3.2 Оператори порівняння

Оператор	Значення
>	більше
<	менше
>=	більше або дорівнює
<=	менше або дорівнює
==	дорівнює (порівняння)
!=	не дорівнює

3.3 Логічні та побітові оператори (*bitwise*)

	логічний	побітовий
AND	&&	&
OR	||	|
NOT	!	~

Визначення

Логічні операторы (&&, ||, !) розглядають кожне число **цілком** як **істину або хибу**: будь-яке число, **відмінне від нуля**, є **істиною** (навіть від'ємне), а **єдине** хибне число — **нуль**. Результат — 1 (істина) або 0 (хиба).
Побітові операторы (&, |, ~) натомість порівнюють **кожен окремий біт** першого числа з **відповідним бітом** другого, **у стовпчик**.

```
char a = 5;
char b = 10;
char c = a && b;
char d = a & b;
```

Приклад

Два різні результати.

- `c = a && b`: 5 відмінне від нуля, отже істина; 10 — істина; істина AND істина дає істину: **`c` дорівнює `1`**.
- `d = a & b`: машина записує числа у двійковому вигляді й робить AND **стовпчик за стовпчиком**:

```
  a = 5   0000 0101
 b = 10   0000 1010
-----------------------
  a & b   0000 0000 → 0, хиба
```

У жодному стовпчику немає двох одиниць, тому **`d` дорівнює `0`**. Ті самі числа, різні оператори: 1 проти 0. Числа 5 і 10 вибрано навмисно, бо одиниці в них стоять на **зовсім різних місцях**, і різниця одразу помітна.
Якщо ж `b` дорівнює **`14`**:

```
  a = 5   0000 0101
 b = 14   0000 1110
-----------------------
  a & b   0000 0100 → 4
```

`a && b` завжди дає 1, а `a & b` дає **`4`** (що, бувши відмінним від нуля, як умова означає істину). Це два **зовсім різні** обчислення.

3.4 Побітовий AND у програмі для 595

Побітовий AND ми використовуємо в програмі, що під'єднує **74HC595** до Arduino (конспекти A02 і A03). Усередині `for`, який надсилає 8 бітів, є:

```
for (int i = 0; i < 8; i++)
{
  digitalWrite(SER, d & (1 << i));
  delay(1);
  carica();
}
```

SER — це вивід 3, **провід даних**; d — число, яке надсилаємо. `1 << i` — це число 1, **зсунуте вліво** на `i` позицій: на кожному кроці одиниця **пересувається** на одне місце і **«виловлює»** інший біт d. Якщо цей біт 1, `d & (1 << i)` відмінне від нуля (істина), і лінія **SER стає високою**; якщо 0, результат 0, і лінія стає **низькою**.

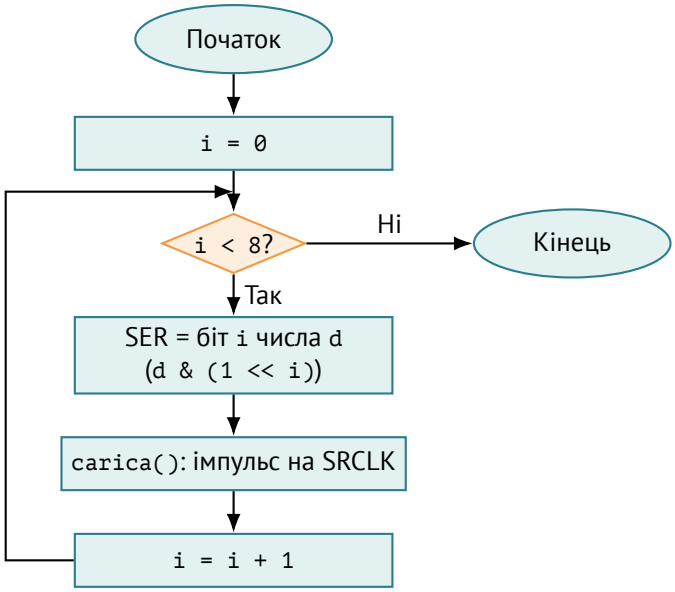


Рис. 1: Цикл `for`, що надсилає 8 бітів d у 595.

4 Конструкції: `if`, `if … else`, `for`, `while`

4.1 Вибір: `if` і `if … else`

Визначення

`if` дозволяє зробити **вибір**: у дужках пишуть **умову**, яка може бути істинною або хибною (наприклад, `a > b`, `a == b`). Код у фігурних дужках виконується **лише якщо** умова **істинна**; якщо хибна, його **пропускають**.

З **`if … else`** є **два шляхи**: якщо умова істинна, виконується перший блок, **інакше** (`else`) — другий.

```
if (a > b)
{
  // виконується, якщо умова істинна
}
else
{
  // виконується, якщо умова хибна
}
```

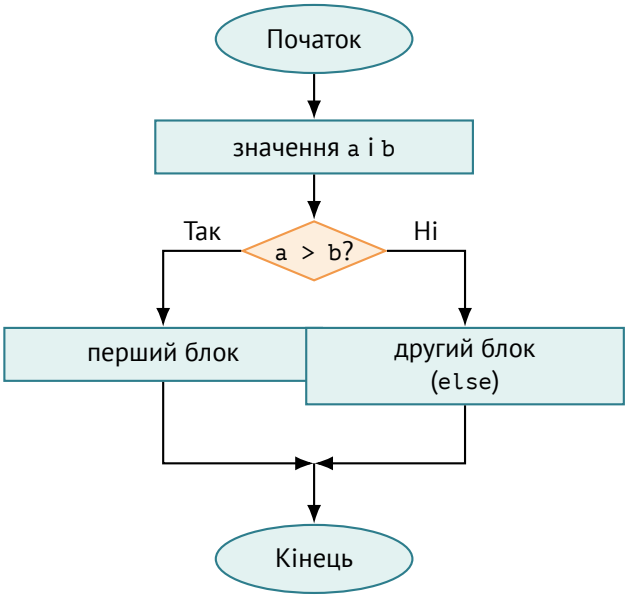


Рис. 2: `if ... else`: два шляхи, проходимо лише один.

4.2 Повторення: `for` і `while`

Визначення

`for` і **`while`** служать для того, щоб **повторювати** частину коду: замість писати її 100 разів, кажемо Arduino повторити її 100 разів. Вони роблять **одне й те саме**.

`for` має три частини в дужках, розділені крапкою з комою: **початкова** **умова** (`int i = 0`), **умова продовження** (`i < 10`) і **модифікатор**, тобто як `i` наближається до мети (`i++`). У фігурних дужках — код, який треба повторити.

```
for (int i = 0; i < 10; i++)
{
  // код, який повторюємо 10 разів
}
```

У **`while`** ті самі три частини стоять у трьох різних місцях: змінну створюють **до** циклу, у дужках лише **умова**, а **модифікатор** пишуть **у кінці**, перед закриттям фігурної дужки (якщо його забути, цикл ніколи не закінчиться).

```
int i = 0;
while (i < 10)
{
  // код, який повторюємо 10 разів
  i++;
}
```

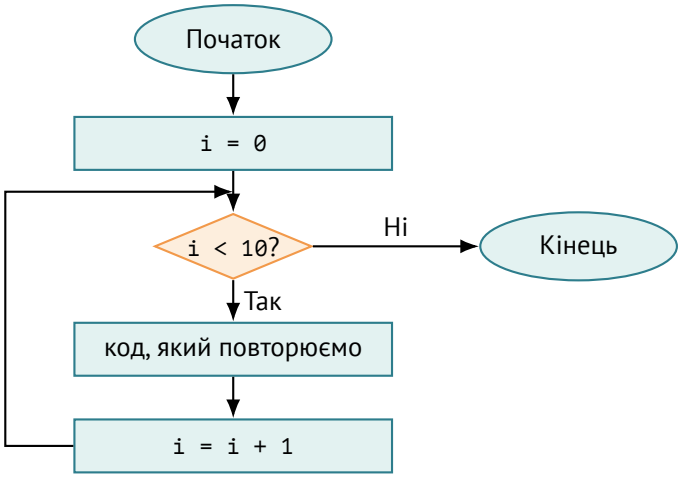


Рис. 3: Та сама блок-схема підходить і для `for`, і для `while` вище.

Примітка

Що вибрати? Можна користуватися тим, що більше подобається: `for` трохи **компактніший**. Зазвичай **`for`** використовують, коли **заздалегідь** відомо, скільки разів повторювати (блимнути світлодіодом 10 разів), а **`while`** — коли **невідомо** і треба продовжувати, **доки щось не станеться**, наприклад доки показання **датчика** не скаже зупинитися.

5 Функції

Єдина справжня новинка цього року — **функції** (*funzioni*).

Визначення

Функція — це **програма в програмі**: група інструкцій у фігурних дужках з **ім'ям**, яку можна **викликати** скільки завгодно разів. Її можна назвати також підпрограмою або *макросом*, але правильна назва — функція.

5.1 Найпростіша форма

Блимання світлодіода на виводі 7 — це чотири рядки. Беремо їх у фігурні дужки і даємо ім'я `lampeggio` («блимання»). Перед ім'ям пишуть **тип значення, що повертається**: тут ми нічого не повертаємо, тому **`void`**. У дужках мали б бути **параметри**: тут жодного.

```
void lampeggio()
{
  digitalWrite(7, HIGH);
  delay(500);
  digitalWrite(7, LOW);
  delay(500);
}
```

У `loop` (який, якщо придивитися, теж є функцією) досить написати **`lampeggio();`**: замість цього Arduino виконає чотири рядки. Написати тричі — світлодіод блимне тричі; помістити в `for` — блимне стільки разів, скільки треба.

5.2 Параметри: передавання за значенням

У дужках можна поставити **змінні** — **параметри**, які заповнюються числами, що ми передаємо під час виклику. Замість сталого 7 використовуємо `x`:

```
void lampeggio(int x)
{
  digitalWrite(x, HIGH);
  delay(500);
  digitalWrite(x, LOW);
  delay(500);
}
```

Тепер `lampeggio(7);` блимає світлодіодом на виводі 7, а `lampeggio(11);` — на виводі 11. Параметри розділяють **комою**, і їх може бути скільки завгодно (`int x, int y, int z, int w`...). Це називається **передавання за значенням** (*passaggio per valore*), бо число (7, 11) **копіюється** у змінну `x` **щоразу**, коли викликаємо функцію.

5.3 Повернути результат: `return`

Визначення

Функція може наприкінці **повернути** значення: дані, результат обчислення. Замість `void` пишуть **тип** значення, що повертається (`int`, `float`, `char`...), а перед кінцем — **`return`** і саме значення.

Приклад із дошки, навмисно простий: функція завжди повертає 3.

```
int lampeggio(int x)
{
  digitalWrite(x, HIGH);
  delay(500);
  digitalWrite(x, LOW);
  delay(500);
  return 3;
}

void loop()
{
  // 7 потрапляє в x; повернене 3 потрапляє в w
  int w = lampeggio(7);
  lampeggio(11);
}
```

Після `w = lampeggio(7);` **w** **дорівнює** **3**: це значення, яке повернула функція і яке скопійовано у змінну. Значення рухаються у **двох** **напрямках**: від виклику до функції (7, що йде в `x`) і від функції до виклику (3, що йде в `w`). Справжній приклад — **гіпотенуза** з конспекту A03: функція отримує два катети і повертає результат обчислення.

Примітка

Чому це зручно. Складну формулу (навіть з іншого предмета) пишуть **один** **раз** у функції; потім їй передають початкові значення, а вона повертає результат. Обчислення **автоматизується**, і ймовірність помилки менша.

6 Що повторити до контрольної

- **pinMode**, **digitalWrite**, **digitalRead**: параметри, великі літери, змінна для прочитаного значення.
- **=** копіює справа наліво; **==** порівнює. Імена змінних: без символів, не починаються з цифри.
- Арифметичні оператори і **модуль**: `5 / 2` = 2, `5 % 2` = 1, `100 % 49` = 2, парні й непарні.
- Оператори **порівняння** (>, <, >=, <=, ==, !=).
- **Логічні** проти **побітових**: `5 && 10` = 1, `5 & 10` = 0, `5 & 14` = 4; `d & (1 << i)` у 595.
- **if**, `if … else`, **for**, **while**: як пишуться і коли використовуються.
- **Функції**: `void`, параметри, **передавання** **за** **значенням**, **return**.
- Конспекти: **A01,** **A01bis,** **A02,** **A03** і цей A04.

Глосарій: італійські слова, які вживають на уроці

Italiano	**Українською**
piedino	вивід (пін)
ingresso / uscita	вхід / вихід
acceso / spento	увімкнено / вимкнено
assegnazione	присвоєння
confronto	порівняння
variabile	змінна
modulo, resto	модуль, остача
pari / dispari	парне / непарне
vero / falso	істина / хиба
bit a bit	побітовий
ciclo	цикл
funzione	функція
parametro	параметр
passaggio per valore	передавання за значенням
lampeggio	блимання
verifica	контрольна робота